

ARPegio: Detección No Supervisada de Anomalías L2 en LAN IEEE 802.3 con Priorización de Alertas

Daniel Alonso Gracia Pinto
Redes de Computadores – Grupo 3
Universidad Nacional de Colombia
Bogotá, Colombia
dagraciap@unal.edu.co

Abstract—Institutional IEEE 802.3 LANs are the operational backbone of critical services, yet they remain largely blind to early-stage threats that emerge below the IP layer. Attacks such as ARP spoofing, MAC flooding, silent port scanning, and ARP reconnaissance manifest as subtle deviations in otherwise legitimate Layer 2 traffic—deviations that signature-based intrusion detection systems consistently fail to capture. Existing Layer 2 security solutions either require active agents deployed on every network node or depend on exhaustively labeled attack datasets, rendering them impractical for real institutional environments. This paper proposes an unsupervised anomaly detection system anchored at the Ethernet frame level, built around a lightweight autoencoder that learns a behavioral baseline of normal traffic without any labeled data. A statistical threshold calibration mechanism controls the false-positive rate without requiring attack samples, and an automatic alert prioritization engine ranks detected anomalies by severity, frequency, and node impact to mitigate operator alert fatigue. A web-based operational platform (*ARPegio*) integrates these components into an analyst-facing workflow comprising monitoring, prioritized triage, interactive laboratory replay, and an optional AI-assisted agent in preventive and reactive modes. The system is evaluated on traffic captured in a controlled IEEE 802.3 LAN emulated with GNS3, comprising four Layer 2 attack families—ARP spoofing, MAC flooding, port scanning, and ARP reconnaissance—and periods of legitimate baseline traffic. From two victim-side captures, 959 temporal windows of native Ethernet features were extracted and labeled for evaluation; model training uses normal traffic only. Results demonstrate the viability of passive, label-free monitoring for proactive IEEE 802.3 LAN cybersecurity, the operational usability of an integrated analyst platform, and highlight the open research gap in Layer 2-anchored unsupervised network intrusion detection.

Index Terms—Anomaly detection, unsupervised learning, autoencoder, IEEE 802.3, Ethernet, ARP spoofing, NIDS, alert prioritization, decision support, Layer 2 security.

Resumen—Las redes LAN IEEE 802.3 institucionales constituyen el núcleo operativo de servicios críticos, pero permanecen en gran medida ciegas ante amenazas tempranas que se originan por debajo de la capa IP. Ataques como ARP spoofing, MAC flooding, escaneo silencioso de puertos y reconocimiento ARP se manifiestan como desviaciones sutiles en el tráfico legítimo de capa 2, desviaciones que los sistemas de detección de intrusos basados en firmas no logran capturar consistentemente. Las soluciones de seguridad existentes para capa 2 exigen agentes activos en cada nodo o dependen de conjuntos de datos de ataques exhaustivamente etiquetados, lo que las hace inviables en entornos institucionales reales. Este artículo propone un sistema de detección de anomalías no supervisado anclado en la trama Ethernet, construido sobre un *autoencoder* ligero que aprende un perfil de normalidad del tráfico sin requerir datos etiquetados.

Un mecanismo de calibración estadística del umbral controla la tasa de falsos positivos sin necesidad de muestras de ataque, y un motor automático de priorización de alertas clasifica las anomalías detectadas según severidad, frecuencia e impacto en el nodo para mitigar el *alert fatigue* del operador. Una plataforma web de operación (*ARPegio*) integra estos componentes en un flujo orientado al analista: monitoreo, *triage* priorizado, laboratorio interactivo y un agente de asistencia opcional en modos preventivo y reactivo. El sistema se evalúa sobre tráfico capturado en una LAN IEEE 802.3 controlada emulada con GNS3, que incluye cuatro familias de ataque de capa 2—ARP spoofing, MAC flooding, escaneo de puertos y reconocimiento ARP—intercaladas con períodos de tráfico legítimo. A partir de dos capturas en nodos víctima se extrajeron 959 ventanas temporales de *features* Ethernet nativas, etiquetadas únicamente para evaluación; el entrenamiento emplea solo tráfico normal. Los resultados demuestran la viabilidad del monitoreo pasivo sin etiquetas para la ciberseguridad proactiva de redes LAN IEEE 802.3, la utilidad operativa de una plataforma integrada para el analista, e identifican la brecha de investigación abierta en detección no supervisada anclada en capa 2 Ethernet.

Palabras clave—Detección de anomalías, aprendizaje no supervisado, *autoencoder*, IEEE 802.3, Ethernet, ARP spoofing, NIDS, priorización de alertas, soporte a la decisión, seguridad en capa 2.

I. INTRODUCCIÓN

Las redes LAN IEEE 802.3 institucionales constituyen el núcleo operativo de servicios críticos, exigiendo niveles óptimos de seguridad y disponibilidad continua. Sin embargo, los ataques tempranos, tales como el escaneo silencioso de puertos, el reconocimiento ARP, el ARP spoofing y el MAC flooding, se camuflan hábilmente como variaciones sutiles del tráfico legítimo. Estas amenazas logran establecerse en la red antes de que el sistema advierta que está bajo ataque [1]. Ante este panorama adverso, los sistemas de detección de intrusos basados en firmas fallan consistentemente frente a vulnerabilidades desconocidas o amenazas adaptativas, evidenciando la necesidad de transitar hacia enfoques centrados en el comportamiento de la red [2].

A pesar de la urgencia de este problema, persisten brechas concretas en la literatura actual. La primera radica en que los estándares de *feature selection* para plataformas NIDS se encuentran fuertemente anclados en flujos IP, empleando protocolos como NetFlow en las capas 3 y 4. Esta perspectiva ignora sistemáticamente la capa 2 Ethernet, siendo este el

nivel exacto de la arquitectura donde ocurren ataques de suplantación como el ARP spoofing y la saturación de tablas mediante MAC flooding; además, las características basadas en flujos presentan un retraso inherente frente a la captura directa a nivel de paquete [3], [4]. La segunda brecha evidencia que las soluciones de seguridad existentes diseñadas para la capa 2 requieren agentes activos instalados en cada nodo o dependen de vastos conjuntos de datos exhaustivamente etiquetados, exigencias técnicas que las hacen completamente inviables para su despliegue en entornos institucionales reales [5], [6]. Una tercera brecha operativa consiste en la ausencia de interfaces que traduzcan los resultados de detección y priorización en flujos de trabajo utilizables por analistas de seguridad, más allá de métricas desacopladas del contexto de red.

Por consiguiente, no existe en la actualidad una solución que combine simultáneamente cuatro factores fundamentales: una detección no supervisada anclada en la capa 2 IEEE 802.3; un monitoreo verdaderamente pasivo y sin agentes que entrene su modelo utilizando únicamente el tráfico normal de la red; un motor automático que clasifique las amenazas para reducir el *alert fatigue* que abruma a los operadores de seguridad [7]; y una capa de presentación que traduzca esos resultados en un flujo operativo accionable para el analista, más allá de métricas aisladas o informes estáticos. Frente a esta carencia, este artículo propone un *autoencoder* no supervisado diseñado para aprender un perfil robusto de normalidad del tráfico Ethernet y detectar desviaciones significativas, complementado con un motor de priorización basado en la severidad, frecuencia e impacto de las anomalías, y con la plataforma *ARPegio* como interfaz de operación y validación. La elección de un método no supervisado se justifica en la realidad operativa: en una LAN institucional, etiquetar todas las variaciones de tráfico malicioso es una tarea inviable. Si bien la literatura reconoce que los métodos supervisados suelen igualar o superar el rendimiento de los no supervisados en entornos de prueba estructurados, el enfoque no supervisado no se plantea aquí como superior en abstracto, sino como la única alternativa técnica viable frente a la ausencia absoluta de etiquetas en este escenario real [2].

En respuesta a estas limitaciones, el presente artículo introduce cuatro contribuciones específicas. Primero, se plantea una arquitectura de monitoreo pasivo que captura tramas Ethernet en un punto de visibilidad del segmento LAN—análogo a un puerto espejo (*SPAN*)—sin requerir agentes en los nodos víctima. Segundo, se desarrolla un modelo de detección de anomalías no supervisado fundamentado en un *autoencoder*, el cual emplea una calibración de umbral estadística sin depender de datos etiquetados. Tercero, se integra un motor de priorización automático de alertas configurado según la severidad, frecuencia y el nivel de impacto en el nodo afectado. Cuarto, se diseña e implementa la plataforma web *ARPegio*, que articula monitoreo, *triage* priorizado, laboratorio interactivo con inferencia en el cliente y asistencia del operador mediante un agente conversacional en modos preventivo y reactivo, cerrando el ciclo entre la evaluación experimental y la decisión en el SOC. El diseño propuesto se fundamenta en criterios estrictos que

guían toda la metodología: la resiliencia ante la manipulación adversaria para evitar la evasión del modelo [4], la viabilidad computacional para operar ágilmente en entornos reales [8], y la reproducibilidad basada en el rigor experimental [9]. El resto del artículo se organiza de la siguiente manera: la Sección II revisa los trabajos relacionados; la Sección III describe la metodología propuesta; la Sección IV detalla la configuración experimental; la Sección V presenta el diseño de la plataforma web de operación; la Sección VI expone y discute los resultados; y la Sección VII concluye el trabajo.

II. TRABAJOS RELACIONADOS

A. NIDS no supervisado

El enfoque no supervisado para sistemas de detección de intrusos en redes (NIDS) ha ganado una tracción significativa, pero presenta limitaciones persistentes en términos de generalización y viabilidad según el contexto de despliegue. Como punto de partida en este debate, Gamage y Samarabandu establecen que los métodos no supervisados son viables en la práctica, aunque suelen tener un rendimiento inferior en los *benchmarks* tradicionales frente a los modelos supervisados; no obstante, su verdadera ventaja competitiva radica en contextos operativos que carecen de etiquetas [2]. Posteriormente, Griffin et al. presentan un *ensemble* de *autoencoders* para operar en tiempo real sobre arquitecturas SDN [10]. Aunque efectivo, este modelo presenta una complejidad computacional mayor que la nuestra y asume una infraestructura SDN que rara vez está disponible en una LAN institucional típica. Por su parte, Antunes et al. realizan un riguroso *benchmark* de métodos de *deep learning* para NIDS, evidenciando claras inconsistencias y un bajo rendimiento al momento de detectar ataques de infiltración [11]. Esto subraya la necesidad de un anclaje más profundo en la red. La limitación común de estos trabajos es que operan exclusivamente sobre flujos IP (como NetFlow), ignorando el nivel base donde los ataques internos realmente se originan.

B. LAN institucional

La literatura que aborda entornos de redes institucionales reales es escasa y habitualmente presenta restricciones críticas relacionadas con la escala de la red o la dependencia absoluta de la supervisión de datos. Tufan et al. validan la existencia de esta problemática en entornos de producción mediante pruebas con ataques de reconocimiento en una red institucional real, confirmando la vulnerabilidad del entorno, aunque basan su propuesta en enfoques supervisados que limitan su aplicabilidad autónoma [1]. En un esfuerzo por abordar despliegues de mayor tamaño, Sun et al. proponen el uso de *Segmented Federated Learning* para LANs a gran escala [12]. Si bien este enfoque escala adecuadamente, exige una coordinación estricta entre nodos, lo cual resulta inviable sin una infraestructura federada dedicada. Dentro de estas redes institucionales, el vector de ataque más crítico e ignorado ocurre de manera insidiosa en la capa 2 Ethernet.

C. Detección en capa 2 IEEE 802.3

La investigación enfocada en la capa 2 existe, pero la mayoría de las soluciones dependen de la modificación de protocolos, agentes activos o esquemas estrictamente supervisados. Morsy y Nashat proponen D-ARP para mitigar el *ARP spoofing*, siendo esta una solución activa que altera el comportamiento estándar del protocolo y exige un control total sobre la infraestructura subyacente [5]. Prasad y Chandra combinan técnicas de aprendizaje automático con *device profiling* para lograr un objetivo similar [13]; sin embargo, su enfoque es supervisado y demanda la instalación de aplicaciones cliente o agentes por nodo. El modelo más cercano a nuestra propuesta es el de Sun et al., quienes utilizan un *autoencoder* para procesar actividad ARP sospechosa [14]. Aunque meritorio, dicho trabajo carece de un anclaje explícito al estándar IEEE 802.3 y no incluye un motor de priorización. Finalmente, Sakhawat et al. definen características ARP en una LAN conmutada mediante una solución basada en agentes [6], un obstáculo de implementación que nuestra propuesta supera al basarse netamente en monitoreo pasivo. Ninguno de estos trabajos aborda el problema de la sobrecarga operativa generada por las alertas, el eslabón final que completa la brecha hacia una solución autónoma.

D. Priorización de alertas en SOC's

La priorización de alertas se reconoce ampliamente como un desafío crítico, pero las alternativas existentes en la literatura dependen excesivamente de la intervención humana o conjuntos de datos previamente etiquetados, factores incompatibles con un *pipeline* analítico totalmente automático. Tariq et al. definen la *alert fatigue* como un cuello de botella paralizante en los SOC's y determinan que la severidad, frecuencia e impacto son los criterios definitivos para abordarla [7], premisa que motiva directamente el diseño de nuestro motor de priorización. En paralelo, Aminanto et al. utilizan un *autoencoder* apilado para reducir los falsos positivos antes de la fase de priorización [15]; esto representa una arquitectura combinada alternativa a nuestra propuesta, pero delega el *autoencoder* a un mero rol de filtro y no como el motor de detección principal. De manera similar, Jalalvand et al. estructuran un marco integral de criterios para la priorización en SOC's [16], que si bien fundamenta los parámetros que adopta nuestro motor, está fuertemente orientado a facilitar la toma de decisiones humana en esquemas colaborativos. Por último, Carrera et al. implementan detección *near real-time* combinando múltiples algoritmos no supervisados [17], resultando en una arquitectura sustancialmente más compleja que nuestra propuesta, y que descartamos para nuestro contexto debido al alto *overhead* computacional que introduce en la red.

La literatura existente logra colectivamente avances significativos en modelos de *deep learning*, visibilidad de capa de enlace y la gestión de la fatiga de alertas, pero invariablemente aborda estos retos de forma aislada o asumiendo infraestructuras fuertemente supervisadas o intervenidas. La brecha específica que ningún trabajo previo llena, y que esta investigación resuelve integralmente, es la consolidación

de un sistema NIDS para redes IEEE 802.3 que opere de forma puramente no supervisada, sin requerir agentes en los nodos, que integre de forma nativa un motor de priorización automático para garantizar la viabilidad operativa en entornos reales, y que disponga de una plataforma de operación que haga interpretables y accionables sus resultados para el analista de seguridad.

III. METODOLOGÍA

A. Visión general del sistema

El diseño metodológico propuesto se estructura como un *pipeline* de extremo a extremo compuesto por cuatro etapas estrictamente secuenciales. La primera etapa realiza la captura pasiva del tráfico Ethernet en el segmento protegido, emulando el despliegue de un IDS pasivo con visibilidad de capa 2. La segunda fase ejecuta la extracción y normalización de características nativas de tramas IEEE 802.3. Posteriormente, la tercera etapa detecta las desviaciones mediante un modelo *autoencoder* de aprendizaje no supervisado. Finalmente, la cuarta fase procesa estos eventos a través de un motor de priorización automática de alertas, evaluadas según su severidad, frecuencia e impacto en el nodo. Una quinta capa de operación—la plataforma *ARPEGIO* descrita en la Sección V—expone los resultados del *pipeline* al analista sin alterar el entrenamiento ni la inferencia del modelo. Durante el entrenamiento y la calibración del umbral, ninguna etapa consume etiquetas de ataque; las etiquetas temporales del *dataset* experimental se reservan exclusivamente para la evaluación posterior del rendimiento. La Fig. 1 resume la arquitectura de extremo a extremo.

B. Arquitectura del autoencoder

La detección de desviaciones conductuales se fundamenta en una arquitectura *encoder-decoder* simétrica y no supervisada. El componente *encoder* recibe el vector de características normalizado y, mediante dos capas densas con funciones de activación ReLU, realiza una reducción dimensional progresiva hacia un espacio latente, específicamente con proyecciones $11 \rightarrow 8 \rightarrow 4$ en el *encoder* y simétricas $4 \rightarrow 8 \rightarrow 11$ en el *decoder*; dicho espacio condensa la representación comprimida y fundamental del comportamiento normal de la red LAN. En contraposición, el bloque *decoder* procesa este espacio latente para reconstruir de forma simétrica el vector original en la salida. El modelo se optimiza empleando el error cuadrático medio (*Mean Squared Error*, MSE) como función de pérdida, entrenándose exclusivamente con tráfico legítimo. Debido a que la red aprende únicamente las regularidades estadísticas de la normalidad, la presencia de tráfico anómalo se manifiesta a través de un error de reconstrucción elevado. Este enfoque basado en un *autoencoder* monolítico se elige por su simplicidad analítica y efectividad para establecer líneas base de seguridad sin requerir etiquetas [18], consolidando al MSE como una métrica robusta e interpretable para la cuantificación de anomalías en entornos de red [19]. Asimismo, frente a soluciones complejas basadas en un *ensemble* de múltiples *autoencoders* orientados a redes lógicas [10], la

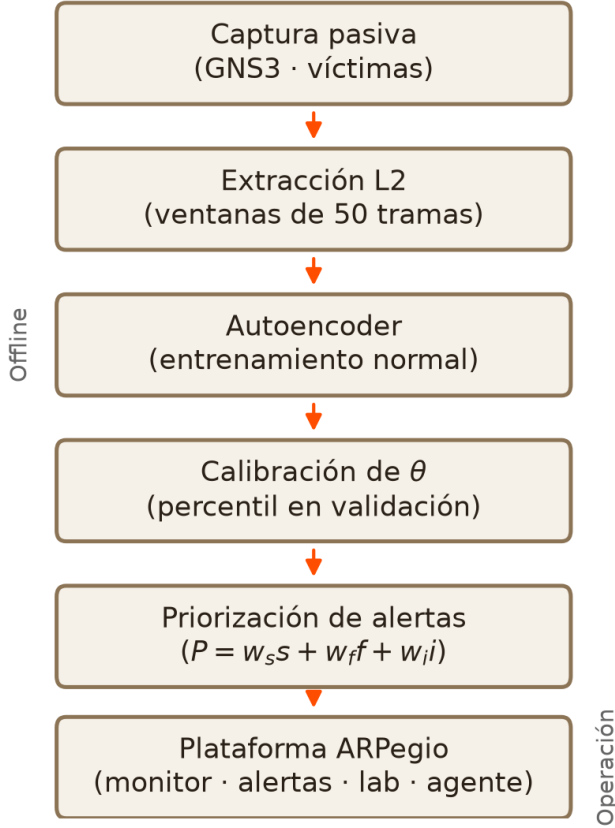


Figure 1. Arquitectura del sistema: captura pasiva, extracción L2, *autoencoder* con calibración de θ , priorización de alertas y plataforma de operación *ARPegio*.

arquitectura propuesta minimiza el *overhead* computacional y resulta óptima y suficiente para representar las dinámicas de la capa de enlace sin comprometer los recursos del sistema.

La adopción de esta arquitectura modular obedece directamente a la necesidad de establecer un marco de trabajo estructurado que garantice un estricto rigor experimental y metodológico en la evaluación de sistemas de detección [9]. Asimismo, la estructuración en un *pipeline* multi-etapa resulta esencial para asegurar la viabilidad computacional del modelo, dado que permite reducir la complejidad general del procesamiento continuo y viabiliza su despliegue ágil en entornos de red reales [8].

C. Adquisición de datos en laboratorio virtual

Para materializar el monitoreo pasivo de capa 2 sin agentes en las víctimas, se emuló un segmento LAN IEEE 802.3 en GNS3 sobre Fedora, interconectando dos nodos Alpine Linux (víctimas), un nodo Kali Linux (atacante) y un *Open vSwitch* (OVS) como conmutador central del segmento 192.168.1.0/24. Las direcciones estáticas asignadas fueron 192.168.1.10 (Alpine-1), 192.168.1.20 (Alpine-2) y

192.168.1.99 (Kali). La conectividad ICMP entre víctimas se verificó antes de iniciar la recolección.

La captura no se realizó en el nodo atacante—escenario poco realista para un IDS de producción—sino en los enlaces físicos simulados entre cada víctima y el OVS, mediante la captura integrada de GNS3 a nivel de hipervisor. Este procedimiento replica un punto de observación neutral del segmento, equivalente operativamente a un puerto espejo (*SPAN*) o a un TAP de red: el sensor observa el tráfico que llega a cada estación protegida, incluyendo tramas ARP, *broadcasts* y tráfico inter-víctima que un host individual no necesariamente genera. Se descartó depender de *tcpdump* dentro de Alpine—sin paquetes disponibles offline—y de configuraciones de *port mirroring* manuales sobre el OVS embebido de GNS3, que resultaron frágiles frente a capturas promiscuas desde Kali.

La sesión experimental se ejecutó como una única captura continua por víctima, con fases alternadas de tráfico legítimo y ataques controlados desde Kali. La Tabla I resume la secuencia. Entre fases se insertaron períodos de tráfico normal (ICMP entre víctimas) para delimitar temporalmente cada familia de ataque en las capturas, de modo que el *dataset* refleje la mezcla realista entre baseline y anomalías observada en operación.

Table I
SECUENCIA EXPERIMENTAL DE RECOLECCIÓN EN GNS3

Fase	Duración	Actividad	Herramienta
1	~3 min	Tráfico normal (ICMP)	<i>ping</i> en Alpine
2	~2 min	ARP spoofing / MITM	<i>arp spoof</i> en Kali
3	~1 min	Pausa normal	<i>ping</i> en Alpine
4	~1 min	MAC flooding	<i>macof</i> en Kali
5	~1 min	Pausa normal	<i>ping</i> en Alpine
6	~3–5 min	Port scan	<i>nmap</i> en Kali
7	~2 min	Reconocimiento ARP	<i>arping</i> en Kali

Las capturas resultantes en ambos nodos víctima contienen $\approx 24\,000$ tramas Ethernet cada una, con una duración total de 34.6 minutos y *link-type* EN10MB, confirmando análisis a nivel de trama IEEE 802.3.

D. Selección de características y preprocesamiento

El vector de *features* se limita a métricas nativas de capa 2 extraídas con Scapy sobre ventanas temporales de 50 tramas consecutivas. Las dimensiones implementadas son: (i) tasa de tramas ARP; (ii) ratio *ARP request/reply*; (iii) entropía de direcciones MAC origen; (iv) conteo de MAC origen y destino únicas; (v) estadísticos de longitud de trama Ethernet (media, desviación estándar, mínimo y máximo); (vi) ratio de tramas broadcast; y (vii) conteo total de paquetes por ventana. Esta selección cubre directamente las familias de ataque evaluadas: el ARP spoofing eleva la tasa ARP con bajo *request/reply*; el MAC flooding concentra la entropía MAC y el número de direcciones origen únicas; el escaneo de puertos reduce la variabilidad de longitud de trama; y el reconocimiento ARP mantiene tasa ARP elevada con patrón distinto al envenenamiento activo. Este anclaje en la capa

de enlace cubre el punto ciego metodológico identificado en sistemas basados exclusivamente en flujos IP [3], [14].

Respecto al preprocesamiento, se aplica una normalización *min-max* para escalar los datos al rango $[0, 1]$. Las ventanas se etiquetan *post-hoc* según el timestamp central de cada ventana y los intervalos temporales de la Tabla I, con cinco clases: tráfico normal, ARP spoofing, MAC flooding, escaneo de puertos y reconocimiento ARP. El entrenamiento del *autoencoder* y la calibración de θ emplean únicamente ventanas de tráfico normal; las etiquetas de ataque se reservan para medir *recall* y *precision* en evaluación. Por política de diseño, no se realiza imputación de valores faltantes; las tramas sin capa Ethernet se descartan al particionar ventanas.

E. Calibración estadística del umbral

El mecanismo de calibración del sistema para la identificación de desviaciones se estructura en tres pasos analíticos. Primero, una vez finalizado el entrenamiento del *autoencoder*, se calcula el error de reconstrucción sobre un conjunto de validación compuesto íntegramente por tráfico normal que el modelo no ha visto previamente. Este procedimiento permite extraer la distribución empírica del error inherente al comportamiento legítimo de la red. Segundo, el umbral de decisión θ se fija dinámicamente seleccionando un percentil superior de esta distribución empírica, típicamente el percentil 95 o 99. Durante la fase de inferencia, cualquier muestra cuyo error de reconstrucción supere estrictamente a θ se clasifica de forma automática como una anomalía. Este enfoque garantiza la detección sin requerir una sola muestra de ataque en la fase de ajuste [21]. Tercero, la adopción de este límite cuenta con una profunda justificación estadística, dado que el percentil elegido permite controlar de manera matemática y directa la tasa de falsos positivos proyectada sobre el tráfico normal de la LAN [22]. La elección de esta calibración estadística por percentil, en claro contraste con alternativas que emplean umbrales numéricos fijos o algoritmos de *clustering* en el espacio latente, se fundamenta en su alta interpretabilidad probabilística y su nula dependencia de datos etiquetados [20]. El valor paramétrico exacto del percentil utilizado constituye un hiperparámetro que se detalla en la configuración experimental.

F. Motor de priorización de alertas

El motor de priorización evalúa cada alerta generada asignándole un puntaje compuesto derivado de tres dimensiones independientes. La primera dimensión es la severidad, definida como la magnitud del error de reconstrucción que supera el umbral estadístico θ , normalizada al rango $[0, 1]$. La segunda dimensión corresponde a la frecuencia, la cual cuantifica el número de alertas emitidas por un mismo nodo origen dentro de una ventana temporal deslizante. La tercera dimensión evalúa el impacto, clasificando el nodo afectado según su rol jerárquico dentro de la topología de la red, otorgando mayor peso a servidores críticos, seguidos de estaciones cliente y dispositivos IoT. El puntaje de prioridad final se calcula como una suma ponderada de estas tres métricas, con

pesos configurables directamente por el operador de seguridad. En la implementación de referencia se fijaron $w_s = 0,5$, $w_f = 0,3$ y $w_i = 0,2$, con

$$P = w_s \cdot s + w_f \cdot f + w_i \cdot i, \quad (1)$$

donde $s = \min(e/\theta, 1)$ es la severidad (error de reconstrucción e respecto al umbral θ), f es la fracción de ventanas anómalas en una ventana deslizante de $W=10$ por fuente de captura, e i es el impacto operacional. Dado que los nodos del laboratorio son clientes homogéneos (Alpine Linux), el impacto se aproxima mediante pesos fijos por familia de amenaza inferida a partir de *features* L2 (MAC flooding: 1.0; ARP spoofing: 0.9; escaneo de puertos: 0.7; reconocimiento ARP: 0.6; desconocido: 0.5), sin emplear etiquetas en tiempo de inferencia. La familia de amenaza se infiere mediante reglas umbral fijas evaluadas en cascada sobre las mismas *features* L2: (i) si el conteo de MAC origen únicas ≥ 20 , se clasifica como MAC flooding; (ii) si no, y la tasa ARP $\geq 0,5$, entonces ratio petición-respuesta ARP < 1 indica ARP spoofing y, en caso contrario, reconocimiento ARP; (iii) si no, y la desviación estándar de longitud de trama < 5 , se asigna escaneo de puertos; (iv) en cualquier otro caso, la alerta queda como *desconocida*. No se emplea aprendizaje supervisado ni *clustering* en esta etapa: son heurísticas deterministas alineadas con las señales dominantes de cada familia descritas en la subsección de selección de características y preprocesamiento.

El diseño de este mecanismo se fundamenta en las necesidades operativas de los Centros de Operaciones de Seguridad (SOCs), adoptando los criterios definitivos establecidos en la literatura especializada para mitigar la fatiga de alertas [7], [16]. A diferencia de enfoques alternativos que emplean *autoencoders* como filtros previos para reducir falsos positivos antes de la priorización [15], o de arquitecturas *near real-time* que introducen un alto *overhead* computacional en la red [17], el motor propuesto procesa el flujo de manera automatizada sin requerir etiquetas ni intervención humana. Finalmente, la implementación de la ventana temporal deslizante permite correlacionar eventos continuos, facilitando la detección de patrones de ataque sostenidos y adaptativos que, de otro modo, se registrarían como alertas individuales inofensivas de muy baja severidad.

IV. CONFIGURACIÓN EXPERIMENTAL

A. Dataset L2 generado

A partir de las dos capturas en nodos víctima se derivó un conjunto de 959 ventanas etiquetadas mediante el *pipeline* descrito en la Sección III. Cada fila corresponde a una ventana de 50 tramas consecutivas y a las 11 *features* numéricas de capa 2, más marcas temporales de inicio y fin, identificador de la fuente de captura y etiqueta de evaluación. La Tabla II resume la distribución resultante tras combinar ambas perspectivas de captura.

El desbalance hacia MAC flooding refleja la naturaleza de ráfaga del ataque *macof*: ≈ 2.5 s de inundación generaron cientos de ventanas consecutivas con 50 direcciones MAC

Table II
DISTRIBUCIÓN DE VENTANAS ETIQUETADAS (959 EN TOTAL)

Clase	n	Señal L2 dominante
Tráfico normal	116	ICMP baseline
ARP spoofing	6	ARP alto, bajo req./reply
MAC flooding	795	50 MACs únicas, entropía $\approx 3,91$
Escaneo de puertos	36	Longitud de trama uniforme
Reconocimiento ARP	6	ARP rate = 1,0, sondeo

distintas por ventana. Las clases ARP (ARP spoofing y reconocimiento ARP) concentran pocas ventanas pero exhiben separación clara en tasa ARP frente a tráfico normal. La perspectiva de la segunda víctima aportó 75 ventanas de tráfico normal adicionales frente a 41 de la primera víctima, lo que refuerza la línea base de entrenamiento no supervisado.

B. Protocolo de entrenamiento y evaluación

El protocolo experimental distingue explícitamente entre uso de etiquetas en entrenamiento y en evaluación:

- **Entrenamiento:** solo ventanas de tráfico normal.
- **Validación (umbral θ):** subconjunto de ventanas de tráfico normal no vistas en entrenamiento (percentil 95 del error MSE).
- **Evaluación:** conjunto completo con las cinco clases, para medir detección de anomalías por familia de ataque.

Las etiquetas temporales se derivaron del instante central de cada ventana y de los intervalos de la sesión GNS3; no intervienen en el ajuste del *autoencoder*. El entorno de implementación emplea Python 3.12, Scapy, pandas, scikit-learn (*MinMaxScaler*), PyTorch y *uv* para reproducibilidad. El entrenamiento utiliza 33 ventanas de tráfico normal de la primera víctima; ocho ventanas normales restantes de la misma fuente calibran θ (percentil 95). La evaluación se realiza sobre 144 ventanas de la segunda víctima no vistas en entrenamiento. Dado el desbalance extremo del *dataset* completo (795 ventanas de MAC flooding frente a 116 de tráfico normal), se aplicó un submuestreo aleatorio de MAC flooding a 116 ventanas en total—con semilla fija para reproducibilidad—antes de separar por fuente de captura. De ese subconjunto, 62 ventanas pertenecen a la segunda víctima y forman el bloque de inundación MAC del conjunto de prueba, junto con las 75 ventanas normales y las clases minoritarias de la misma perspectiva. Así se evita que las métricas queden dominadas por la ráfaga de MAC flooding sin renunciar a evaluar ataques reales de la captura. El optimizador empleado es Adam con tasa de aprendizaje $lr = 1 \times 10^{-3}$, tamaño de lote de 16 ventanas y 100 épocas de entrenamiento. La ventana deslizante del motor de priorización se fijó en $W = 10$. La implementación se organiza en cuatro componentes funcionales: entrenamiento del *autoencoder*, motor de priorización heurística, emisión de alertas ordenadas y plataforma web de operación. El cuarto componente—*ARPEGIO*, descrito en la Sección V—expone métricas, alertas y escenarios de laboratorio al analista sin depender del entorno de entrenamiento.

V. DISEÑO DE LA PLATAFORMA

Para trasladar el *pipeline* de detección L2 a un entorno operativo comprensible por analistas, se implementó *ARPEGIO*, una aplicación web estática que incorpora las métricas de evaluación, la cola de alertas priorizadas, los puntajes por ventana y los escenarios de laboratorio producidos por el entrenamiento y la priorización. La interfaz no reimplementa el entrenamiento ni la extracción de *features*: actúa como capa de visualización, *triage* y experimentación que cierra el ciclo entre la evaluación offline y la toma de decisiones del SOC. La demo pública se despliega en la Web ¹, lo que permite reproducir la experiencia sin instalar componentes de inferencia en el servidor.

A. Arquitectura de la interfaz

La plataforma se organiza en seis vistas navegables desde una barra lateral persistente: **Monitor**, **Red**, **Alertas**, **Laboratorio**, **Agente** y **Ajustes**. Al iniciar, la aplicación carga los resultados del *pipeline* y deriva métricas agregadas—AUC-ROC, umbral θ , matriz de confusión por percentil y resúmenes de alertas—que alimentan la interfaz dinámica. Esta separación entre el procesamiento experimental del modelo y la capa de presentación mantiene el rigor del entrenamiento mientras ofrece interactividad al operador. La Fig. 2 ilustra la vista **Monitor**, punto de entrada del tablero operativo.

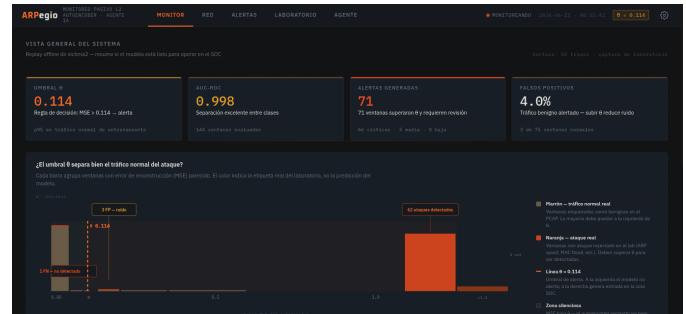


Figure 2. Vista Monitor: indicadores del *pipeline*, distribución del error de reconstrucción y exploración de percentiles de θ .

B. Monitoreo, red y cola de alertas

La vista **Monitor** resume el estado del modelo tras el *replay* offline de la segunda víctima: AUC-ROC, número de alertas, tasa de falsos positivos y un histograma que contrasta el error MSE con la etiqueta real de laboratorio (no con la predicción del modelo). Un control deslizante permite simular distintos percentiles de θ y observar el trade-off TN/FP/FN/TP sin reentrenar.

La vista **Red** representa la topología GNS3 del segmento 192.168.1.0/24—nodos víctima Alpine, nodo atacante Kali y conmutador OVS—junto con una línea temporal de ventanas etiquetadas de la sesión experimental. La Fig. 3 corresponde a este módulo de contexto espacio-temporal.

¹<https://vethariel.github.io/ARPEGIO/>

La vista **Alertas** materializa la salida del motor de priorización: tabla ordenada por puntaje P , filtros por severidad y familia de ataque, y panel de detalle con el vector de *features* L2 de la ventana seleccionada. Las tres alertas de mayor prioridad se destacan también en **Monitor** como punto de entrada al *triage*. La Fig. 4 muestra la cola priorizada y el detalle de una ventana.

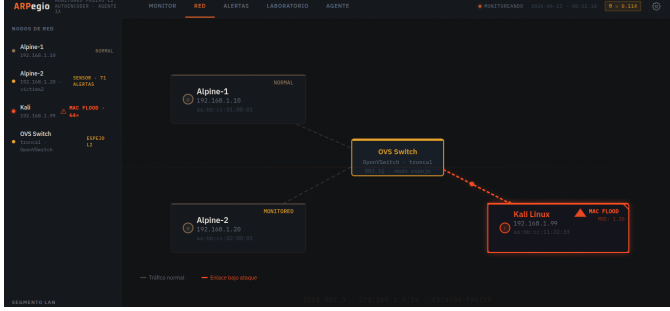


Figure 3. Vista Red: topología del laboratorio y secuencia temporal de ventanas etiquetadas.

SEVERIDAD	FAMILIA	RECURSO	TIEMPO	DETALLE
2.20	CRITICO	MAC Flood	10:00:00	1.20
1.20	CRITICO	MAC Flood	10:00:01	
1.20	CRITICO	MAC Flood	10:00:02	
1.20	CRITICO	MAC Flood	10:00:03	
1.20	CRITICO	MAC Flood	10:00:04	
1.20	CRITICO	MAC Flood	10:00:05	
1.20	CRITICO	MAC Flood	10:00:06	
1.20	CRITICO	MAC Flood	10:00:07	
1.20	CRITICO	MAC Flood	10:00:08	
1.20	CRITICO	MAC Flood	10:00:09	
1.20	CRITICO	MAC Flood	10:00:10	
1.20	CRITICO	MAC Flood	10:00:11	
1.20	CRITICO	MAC Flood	10:00:12	
1.20	CRITICO	MAC Flood	10:00:13	
1.20	CRITICO	MAC Flood	10:00:14	
1.20	CRITICO	MAC Flood	10:00:15	
1.20	CRITICO	MAC Flood	10:00:16	
1.20	CRITICO	MAC Flood	10:00:17	
1.20	CRITICO	MAC Flood	10:00:18	
1.20	CRITICO	MAC Flood	10:00:19	
1.20	CRITICO	MAC Flood	10:00:20	

Figure 4. Vista Alertas: cola priorizada, filtros operativos y detalle de la ventana seleccionada.

C. Laboratorio interactivo con inferencia en el cliente

La vista **Laboratorio** reproduce escenarios reales de la captura—ARP spoofing, MAC flooding, escaneo de puertos, reconocimiento ARP y tráfico tráfico normal—mediante inferencia del *autoencoder* en el navegador. El modelo exportado se ejecuta íntegramente en el cliente, sin enviar tráfico ni vectores de *features* a un servidor remoto. Un control de percentiles recalcula θ al vuelo y compara el MSE reconstruido frente al umbral elegido; el escenario de escaneo de puertos permanece documentado como caso de falso negativo honesto cuando el punto de observación no captura la señal dominante. La Fig. 5 resume esta capacidad de validación interactiva.

D. Agente de asistencia y configuración

La vista **Agente** integra un copiloto conversacional con modelo de lenguaje (*bring your own key*): la credencial permanece en el navegador del operador y no transita por el servidor de la demo. Se distinguen dos modos de operación: **preventivo**—calibración de θ , análisis de falsos positivos y checklist pre-incidente—y **reactivo**—planes de contención L2

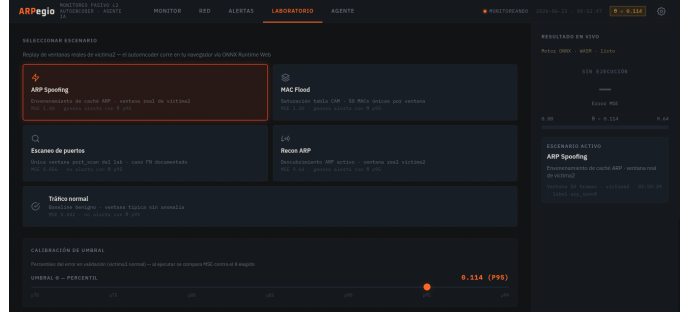


Figure 5. Vista Laboratorio: *replay* de escenarios reales con inferencia en el navegador y calibración interactiva de θ .

y comandos sugeridos para el laboratorio tras una alerta. El agente emite informes estructurados con veredicto, nivel de confianza, recomendación de umbral, escenario de laboratorio sugerido, acciones priorizadas y comandos de verificación. Además, puede simular distintos percentiles de θ , consultar alertas individuales, listar la cola priorizada y recuperar el perfil de escenarios del laboratorio a partir del estado vigente del sistema. Un diálogo de seguimiento permite refinar las respuestas; la interfaz presenta notación matemática y comandos de forma legible para el operador. Accesos contextuales desde **Monitor**, **Alertas** y **Laboratorio** abren el agente con la situación ya cargada. La ejecución remota de mitigaciones en GNS3 no está automatizada: el agente asiste al operador, pero la contención sigue siendo manual. La Fig. 6 muestra el flujo de informe en modo reactivo.

La vista **Ajustes** centraliza la credencial del agente, parámetros de modelo de solo lectura (arquitectura $11 \rightarrow 8 \rightarrow 4 \rightarrow 8 \rightarrow 11$, AUC, θ activo) y acceso a la documentación del proyecto y a las capturas del laboratorio. La Fig. 7 corresponde a este panel.

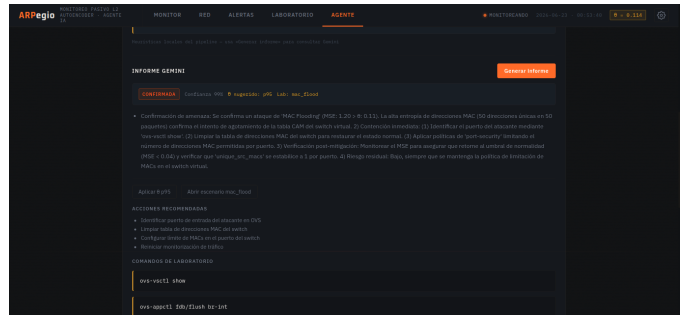


Figure 6. Vista Agente: modos preventivo y reactivo, informe estructurado y diálogo de seguimiento con consulta al estado del sistema.

VI. RESULTADOS Y DISCUSIÓN

A. Entrenamiento del autoencoder

El modelo convergió en 100 épocas con pérdida MSE final de $3,9 \times 10^{-2}$ sobre 33 ventanas de tráfico normal de la primera víctima. La Fig. 8 muestra la curva de entrenamiento monótonamente decreciente, sin sobreajuste aparente dado el reducido tamaño del conjunto de entrenamiento no supervisado.

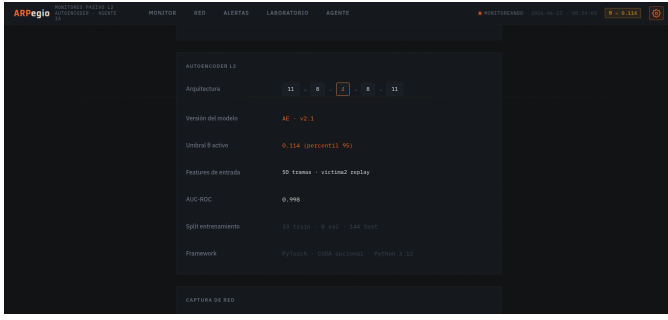


Figure 7. Vista Ajustes: configuración del agente, metadatos del *autoencoder* y acceso a documentación del proyecto.

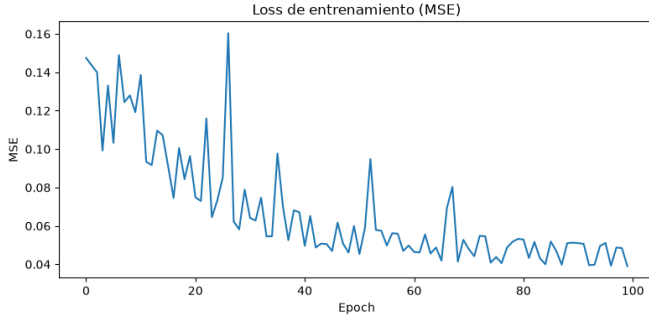


Figure 8. Curva de pérdida MSE durante el entrenamiento (100 épocas).

B. Calibración del umbral y separabilidad

El umbral $\theta = 0,114$ se obtuvo como percentil 95 del error MSE sobre ocho ventanas normales de validación (perspectiva de la primera víctima, no vistas en entrenamiento). La Fig. 9 contrasta la distribución de errores normales frente a ventanas de ataque en la segunda víctima: las familias ARP y MAC flooding quedan claramente separadas del tráfico legítimo.

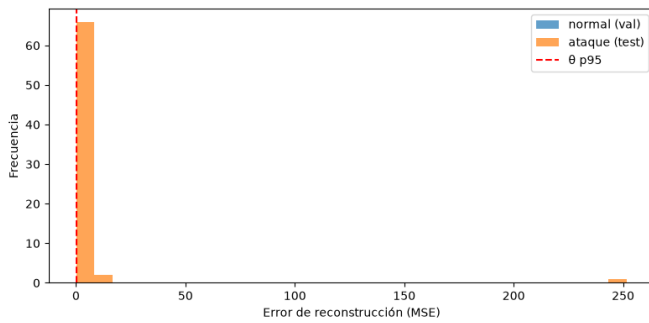


Figure 9. Distribución del error de reconstrucción: validación normal vs. ataques en la segunda víctima; línea vertical en θ .

C. Detección binaria y AUC

Sobre 144 ventanas de prueba (perspectiva de la segunda víctima), la clasificación binaria normal/ataque alcanzó precisión, *recall* y F1 de 0.97 en ambas clases (exactitud global 97%). El AUC-ROC fue 0,9986, indicando excelente capaci-

dad discriminativa del error de reconstrucción como estadístico de anomalía.

D. Detección por familia de ataque

La Tabla III resume la tasa de detección por etiqueta de evaluación. Tres de las cuatro familias de ataque L2 alcanzaron detección perfecta en la segunda víctima. No obstante, dado que ARP spoofing cuenta con únicamente tres ventanas en el conjunto de prueba, este resultado debe interpretarse con cautela estadística. La ausencia de detección en escaneo de puertos (0/1) refleja la baja visibilidad de este ataque desde la perspectiva de la segunda víctima, donde solo una ventana fue etiquetada para esta clase; no constituye un fallo del modelo sino una limitación del punto de observación (36 ventanas en el *dataset* completo frente a 795 de MAC flooding).

Table III
DETECCIÓN POR FAMILIA DE ATAQUE (SEGUNDA VÍCTIMA, $\theta = 0,114$)

Clase de tráfico	Detectadas	Total	Tasa (%)
MAC flooding	62	62	100.0
ARP spoofing	3	3	100.0
Reconocimiento ARP	3	3	100.0
Escaneo de puertos	0	1	0.0
Tráfico normal (FP)	3	75	4.0

Los tres falsos positivos sobre 75 ventanas normales (4%) se sitúan dentro del margen esperado para un percentil 95 de calibración sobre un subconjunto pequeño de validación.

E. Motor de priorización

Tras aplicar el motor de la ecuación (1) sobre las 71 alertas generadas en la segunda víctima—62 de MAC flooding, tres de ARP spoofing, tres de reconocimiento ARP y tres falsos positivos sobre tráfico normal—las ventanas de MAC flooding persistente obtuvieron puntaje máximo ($P = 1,0$) por combinar severidad, frecuencia deslizante e impacto operacional elevado. Las alertas ARP aparecieron con menor frecuencia acumulada pero mantuvieron prioridad alta gracias a $s \approx 1$ e impacto $i \geq 0,6$. El ordenamiento descendente por P permite al operador atender primero ráfagas sostenidas de inundación MAC antes que eventos aislados de baja severidad, mitigando el *alert fatigue* [7]. La Tabla IV muestra alertas representativas de la segunda víctima con los valores de s , f , i y P calculados por la ecuación (1).

Table IV
EJEMPLO DE PRIORIZACIÓN DE ALERTAS (SEGUNDA VÍCTIMA, $\theta = 0,114$, $W = 10$)

#	Amenaza inferida	Etiqueta real	s	f	i	P
1	MAC flood	MAC flood	1.00	1.00	1.0	1.00
2	MAC flood	MAC flood	1.00	0.90	1.0	0.97
3	ARP spoof	ARP spoof	1.00	0.30	0.9	0.77
4	Recon. ARP	Recon. ARP	1.00	0.10	0.6	0.65
5	Desconocida	Normal (FP)	1.00	0.10	0.5	0.63

Las filas 1–2 ilustran una ráfaga sostenida de MAC flooding con $f \rightarrow 1$. La fila 3 corresponde a ARP spoofing con

menor persistencia ($f = 0,3$) pero impacto elevado ($i = 0,9$). La fila 5 documenta un falso positivo sobre tráfico normal cuya prioridad relativamente alta ($P = 0,63$) refleja severidad máxima pese a baja frecuencia e impacto moderado.

F. Validación operativa de la plataforma

Los resultados cuantitativos anteriores se contrastan en la interfaz descrita en la Sección V. La vista **Monitor** reproduce el AUC-ROC de 0,999, el umbral θ calibrado, el volumen de alertas y la tasa de falsos positivos del 4%, junto con un histograma equivalente a la Fig. 9; un control de percentiles permite explorar el trade-off TN/FP/FN/TP sin reentrenar. La vista **Alertas** ordena las 71 ventanas anómalas por puntaje P y destaca las tres de mayor prioridad, coherente con el comportamiento del motor de priorización (MAC flooding con $P = 1,0$ frente a eventos ARP menos persistentes). La vista **Laboratorio** reproduce la decisión binaria del *autoencoder* en el cliente y documenta de forma explícita el falso negativo de escaneo de puertos desde la segunda víctima (Fig. 5). El módulo **Agente** complementa esta validación con asistencia preventiva y reactiva, accesible de forma contextual desde las demás vistas (Figs. 2, 4 y 6). En conjunto, la plataforma confirma que las métricas del experimento son trazables y accionables para el analista, sin sustituir la evaluación estadística de las subsecciones precedentes.

G. Limitaciones

El tamaño reducido de clases minoritarias (ARP spoofing, reconocimiento ARP y escaneo de puertos) limita la generalización estadística. La validación cruzada por víctima (primera víctima $\rightarrow$ segunda víctima) demuestra transferencia entre perspectivas de captura, pero no sustituye un despliegue en producción con mayor diversidad topológica.

En el plano de la plataforma, la inferencia en el navegador y la carga estática de resultados del *pipeline* la orientan hacia demostración, capacitación y *triage* sobre capturas ya procesadas, no hacia monitoreo en tiempo real de alto caudal. El agente de asistencia opera con credencial aportada por el operador y no ejecuta mitigaciones de forma autónoma en el laboratorio: su valor reside en la interpretación y la propuesta de acciones, no en la orquestación remota de contención. La integración con flujos SOC empresariales (SIEM, *ticketing*, autenticación institucional) permanece como trabajo pendiente.

VII. CONCLUSIONES

Este trabajo consolidó un *pipeline* reproducible de detección de anomalías L2 en redes LAN IEEE 802.3: captura pasiva en laboratorio GNS3, extracción de 11 *features* Ethernet nativas, entrenamiento no supervisado con *autoencoder* y calibración estadística del umbral sin muestras de ataque. Sobre tráfico de la segunda víctima no visto en entrenamiento, el sistema alcanzó 97% de exactitud binaria y AUC-ROC de 0,999, detectando por completo MAC flooding, ARP spoofing y reconocimiento ARP—si bien ARP spoofing y reconocimiento ARP cuentan con muestras de prueba reducidas ($n = 3$ cada

una)—, con una tasa de falsos positivos del 4% sobre tráfico normal.

La integración de un motor de priorización por severidad, frecuencia e impacto ordenó 71 alertas sin intervención humana ni etiquetas en inferencia, demostrando que el *alert fatigue* puede mitigarse incluso con un subconjunto acotado de validación. Más allá del núcleo analítico, la plataforma *ARPEGIO* materializa estos hallazgos en una experiencia operativa de extremo a extremo: el analista supervisa métricas y separabilidad del modelo, explora el compromiso del umbral de forma interactiva, recorre la cola priorizada con contexto topológico, reproduce escenarios de laboratorio con inferencia en el cliente y recibe asistencia del agente en modos preventivo y reactivo. Esta capa de presentación no sustituye la evaluación cuantitativa de las subsecciones anteriores, sino que la hace accionable para un perfil de usuario no especializado en aprendizaje automático.

Los resultados confirman que el monitoreo pasivo anclado en tramas IEEE 802.3 es viable para alertar tempranamente sobre ataques de suplantación y saturación en LAN institucionales simuladas. La combinación de detección no supervisada, priorización heurística e interfaz de operación constituye una respuesta coherente a la brecha identificada en la literatura: sistemas L2 sin agentes en los nodos, sin dependencia de etiquetas en entrenamiento y con un camino claro desde la anomalía detectada hasta la decisión del analista.

Como líneas futuras se identifican: ampliar capturas para escaneo de puertos y clases minoritarias; incorporar alimentación en tiempo real o *streaming* de ventanas L2 a la plataforma; automatizar la ejecución supervisada de mitigaciones sugeridas por el agente en el laboratorio GNS3; integrar autenticación institucional y exportación hacia SIEM; y validar el motor de priorización con roles heterogéneos (servidor/cliente/IoT) en topologías más amplias. En conjunto, el trabajo aporta evidencia experimental y un prototipo operativo que acercan la detección de anomalías en capa de enlace a un escenario institucional verificable y comunicable.

REFERENCES

- [1] E. Tufan, C. Tezcan, and C. Acartürk, “Anomaly-Based Intrusion Detection by Machine Learning: A Case Study on Probing Attacks to an Institutional Network,” *IEEE Access*, vol. 9, pp. 50078–50092, 2021.
- [2] S. Gamage and J. Samarabandu, “Deep learning methods in network intrusion detection: A survey and an objective comparison,” *Journal of Network and Computer Applications*, vol. 169, p. 102767, 2020.
- [3] M. Sarhan, S. Layeghy, and M. Portmann, “Towards a Standard Feature Set for Network Intrusion Detection System Datasets,” *Mobile Networks and Applications*, vol. 27, pp. 357–370, 2022.
- [4] K. He, D. D. Kim, and M. R. Asghar, “Adversarial Machine Learning for Network Intrusion Detection Systems: A Comprehensive Survey,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 538–566, First Quarter 2023.
- [5] S. M. Morsy and D. Nashat, “D-ARP: An Efficient Scheme to Detect and Prevent ARP Spoofing,” *IEEE Access*, vol. 10, pp. 49142–49153, 2022.
- [6] D. Sakhawat, A. N. Khan, M. Aslam, and A. T. Chronopoulos, “Agent-based ARP cache poisoning detection in switched LAN environments,” *IET Networks*, vol. 8, no. 1, pp. 67–73, 2019.
- [7] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, “Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities,” *ACM Computing Surveys*, vol. 57, no. 9, 2025.

- [8] M. Injadat, A. Moubayed, A. B. Nassif, and A. Shami, "Multi-Stage Optimized Machine Learning Framework for Network Intrusion Detection," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1803–1816, Jun. 2021.
- [9] Z. Yang, X. Liu, T. Li, D. Wu, J. Wang, Y. Zhao, and H. Han, "A systematic literature review of methods and datasets for anomaly-based network intrusion detection," *Computers & Security*, vol. 116, p. 102675, 2022.
- [10] L. Griffin, Z. Yang, Y. Song, S. Gao, A. Hu, and B. Xiao, "Griffin: Real-Time Network Intrusion Detection System via Ensemble of Autoencoder in SDN," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2269–2282, Sep. 2022.
- [11] M. Antunes, L. Oliveira, A. Seguro, J. Veríssimo, R. Salgado, and T. Murteira, "Benchmarking Deep Learning Methods for Behaviour-Based Network Intrusion Detection," *Informatics*, vol. 9, no. 1, p. 29, 2022.
- [12] Y. Sun, H. Esaki, and H. Ochiai, "Adaptive Intrusion Detection in the Networking of Large-Scale LANs With Segmented Federated Learning," *IEEE Open Journal of the Communications Society*, vol. 2, pp. 102–112, 2021.
- [13] A. Prasad and S. Chandra, "Defending ARP Spoofing-based MitM Attack using Machine Learning and Device Profiling," in *2022 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, 2022, pp. 978–983.
- [14] Y. Sun, H. Ochiai, and H. Esaki, "Suspicious ARP Activity Detection and Clustering Based on Autoencoder Neural Networks," in *IEEE Consumer Communications & Networking Conference (CCNC)*, 2022.
- [15] M. E. Aminanto, T. Ban, R. Isawa, T. Takahashi, and D. Inoue, "Threat Alert Prioritization Using Isolation Forest and Stacked Auto Encoder With Day-Forward-Chaining Analysis," *IEEE Access*, vol. 8, pp. 217977–217986, 2020.
- [16] F. Jalalvand, M. B. Chhetri, S. Nepal, and C. Paris, "Alert Prioritisation in Security Operations Centres: A Systematic Survey on Criteria and Methods," *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, 2024.
- [17] F. Carrera et al., "Combining Unsupervised Approaches for Near Real-Time Network Traffic Anomaly Detection," *Applied Sciences*, vol. 12, no. 3, p. 1759, 2022.
- [18] X. Li, W. Chen, Q. Zhang, and L. Wu, "Building Auto-Encoder Intrusion Detection System Based on Random Forest Feature Selection," *Computers & Security*, vol. 95, p. 101851, 2020.
- [19] Y. Song, C. Hyun, and Y. Cheong, "Self-Supervised Anomaly Detection with Neural Transformation Learning for Log-Based Intrusion Detection," *IEEE Access*, vol. 9, pp. 53127–53143, 2021.
- [20] N. Maleki, M. Zeinali, and S. T. Hasson, "A Kind of Threshold-Based Anomaly Detection in Autoencoder," in *2021 5th International Conference on Pattern Recognition and Image Analysis (IPRIA)*, 2021, pp. 1–6.
- [21] S. Zavrak and M. İskefiyeli, "Anomaly-Based Intrusion Detection From Network Flow Features Using Variational Autoencoder," *IEEE Access*, vol. 8, pp. 108346–108358, 2020.
- [22] H. Zhang, X. Li, T. Liu, and J. Gu, "Threshold Optimization in Autoencoder-Based Anomaly Detection for Network Traffic," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1753–1765, Jun. 2023.